

OSSP PRACTICAL WEEK 4

2520080012

```
GNU nano 7.2                                     week4_t1.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t children[3];

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    for (int i = 0; i < 3; i++) {
        children[i] = fork();

        if (children[i] == 0) {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);
            printf("Child %d completed. PID = %d\n", i + 1, getpid());

            exit(10 + i);
        }
    }

    printf("All 3 child processes have been created.\n");

    printf("Using wait():\n");
    int status;
    pid_t pid = wait(&status);

    if (WIFEXITED(status)) {
        printf("wait() detected child PID %d finished with exit status %d\n",
            pid, WEXITSTATUS(status));
    }

    printf("Using waitpid():\n");

    for (int i = 0; i < 3; i++) {
        pid = waitpid(children[i], &status, 0);

        if (WIFEXITED(status)) {
            printf("waitpid() detected child PID %d finished with exit status %d\n",
                pid, WEXITSTATUS(status));
        }
    }

    printf("Parent process completed.\n");

    return 0;
}
```

```
root@LAPTOP-0PA61QEN:~# ls /home
alurusaranya
root@LAPTOP-0PA61QEN:~# su alurusaranya
alurusaranya@LAPTOP-0PA61QEN:/root$ ls
ls: cannot open directory '.': Permission denied
alurusaranya@LAPTOP-0PA61QEN:/root$ cd ~
alurusaranya@LAPTOP-0PA61QEN:~$ pwd
/home/alurusaranya
alurusaranya@LAPTOP-0PA61QEN:~$ nano week4_t1.c
alurusaranya@LAPTOP-0PA61QEN:~$ gcc week4_t1.c -o week4_t1
alurusaranya@LAPTOP-0PA61QEN:~$ ./week4_t1
Parent Process Started
Parent PID: 645
All 3 child processes have been created.
Child 1 started. PID = 646, Parent PID = 645
Child 2 started. PID = 647, Parent PID = 645
Using wait():
Child 3 started. PID = 648, Parent PID = 645
Child 1 completed. PID = 646
wait() detected child PID 646 finished with exit status 10
Using waitpid():
waitpid() detected child PID -1 finished with exit status 10
Child 2 completed. PID = 647
waitpid() detected child PID 647 finished with exit status 11
Child 3 completed. PID = 648
waitpid() detected child PID 648 finished with exit status 12
Parent process completed.
alurusaranya@LAPTOP-0PA61QEN:~$ |
```

Task 2:

```
root@LAPTOP-0PA61QEN:~# ls /home
alurusaranya
root@LAPTOP-0PA61QEN:~# su alurursaranya
su: user alurursaranya does not exist or the user entry does not
contain all the required fields
root@LAPTOP-0PA61QEN:~# su alurusaranya
alurusaranya@LAPTOP-0PA61QEN:/root$ ls
ls: cannot open directory '.': Permission denied
alurusaranya@LAPTOP-0PA61QEN:/root$ cd ~
alurusaranya@LAPTOP-0PA61QEN:~$ pwd
/home/alurusaranya
alurusaranya@LAPTOP-0PA61QEN:~$ nano week4_t2.c
alurusaranya@LAPTOP-0PA61QEN:~$ gcc week4_t2.c -o week4_t2
alurusaranya@LAPTOP-0PA61QEN:~$ ./week4_t2
Parent process: PID = 540
Child process: PID = 541
Child PID = 541
Child process terminating...
Parent sleeping...
|
```

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    } else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping...\n");

        sleep(20);

        printf("Parent terminating...\n");
    }

    return 0;
}
```

```

alurusaranya@LAPTOP-0PA61QEN:~$ nano week4_t2.c
alurusaranya@LAPTOP-0PA61QEN:~$ ps -el

```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
4	S	0	1	0	0	80	0	-	5449	-	?	00:00:01	systemd
5	S	0	2	1	0	80	0	-	795	-	hvc0	00:00:00	init-systemd(Ub
0	S	0	6	2	0	80	0	-	799	-	hvc0	00:00:00	init
4	S	0	55	1	0	79	-1	-	12610	-	?	00:00:00	systemd-journal
4	S	0	103	1	0	80	0	-	6354	-	?	00:00:00	systemd-udev
4	S	991	114	1	0	80	0	-	5336	-	?	00:00:00	systemd-resolve
4	S	996	133	1	0	80	0	-	22759	-	?	00:00:00	systemd-timesyn
4	S	0	166	1	0	80	0	-	1061	-	?	00:00:00	cron
4	S	101	167	1	0	80	0	-	2414	-	?	00:00:00	dbus-daemon
4	S	0	176	1	0	80	0	-	4540	-	?	00:00:00	systemd-logind
4	S	0	178	1	0	80	0	-	439026	-	?	00:00:00	wsl-pro-service
4	S	102	190	1	0	80	0	-	55629	-	?	00:00:00	rsyslogd
4	S	0	199	1	0	80	0	-	781	-	tty1	00:00:00	agetty
4	S	0	212	1	0	80	0	-	26758	-	?	00:00:00	unattended-upgr
4	S	0	277	2	0	80	0	-	1668	-	pts/1	00:00:00	login
4	S	0	335	1	0	80	0	-	5081	-	?	00:00:00	systemd
5	S	0	336	335	0	80	0	-	5294	-	?	00:00:00	(sd-pam)
4	S	0	360	277	0	80	0	-	1520	-	pts/1	00:00:00	bash
5	S	0	457	2	0	80	0	-	797	-	?	00:00:00	SessionLeader
5	S	0	459	457	0	80	0	-	801	-	?	00:00:00	Relay(462)
4	S	0	462	459	0	80	0	-	1520	-	pts/0	00:00:00	bash
4	S	0	492	462	0	80	0	-	1702	-	pts/0	00:00:00	su
4	S	1000	495	1	0	80	0	-	5084	-	?	00:00:00	systemd
5	S	1000	496	495	0	80	0	-	5292	-	?	00:00:00	(sd-pam)
4	S	1000	518	492	0	80	0	-	1521	do_wai	pts/0	00:00:00	bash
0	R	1000	553	518	0	80	0	-	2080	-	pts/0	00:00:00	ps

```

alurusaranya@LAPTOP-0PA61QEN:~$ |

```

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    } else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        wait(NULL);

        printf("Child process collected by parent.\n");
        printf("Parent terminating...\n");
    }

    return 0;
}
```